

Étude et analyse de modèles simulant une trajectoire circulaire

Manuel Rocca, Damian Kazberuk, Matteo Morbée,
Romain Liefferinckx, Martin Gouverneur

April 30, 2026

Introduction

La modélisation de systèmes dynamiques implique un compromis entre fidélité physique, coût de calcul et interprétabilité. Ce travail étudie la simulation de la trajectoire d'une bille en mouvement circulaire sur un dispositif expérimental, un système simple mais suffisamment riche pour comparer différentes approches.

Deux familles de méthodes sont considérées. Les modèles physiques explicites (conique et élastique), résolus par intégration numérique. La seconde famille, quant à elle, repose sur l'apprentissage automatique (régression linéaire, MLP) approximent la dynamique à partir de données, sans hypothèses a priori, mais avec une interprétabilité réduite.

L'analyse se penche aussi sur les erreurs de mesure, de modélisation et numériques, ainsi que sur les propriétés de consistance, stabilité et conditionnement. L'objectif est d'évaluer les performances, les limites et le domaine de validité de ces approches du point de vue numérique.

Cadre expérimental

Le dispositif expérimental consiste en une surface élastique en latex tendue sur un cadre circulaire, déformée par une bille suffisamment lourde placée au centre. Une rampe de lancement permet de donner à la bille une vitesse initiale contrôlée. Un visuel est fourni en figure 1.

Modèles physiques

Les modèles physiques reposent sur la mécanique de Lagrange appliquée à une surface de révolution (voir Annexe pour la dérivation complète). La dynamique est réduite à deux coordonnées généralisées (r, θ) ; l'état du système est $\mathbf{q} = (r, \theta, \dot{r}, \dot{\theta}) \in \mathbb{R}^4$.

Modèle conique: La surface est approchée par un cône de pente constante

$$\alpha = \arctan(h/(R - r_{\text{sph}})) \quad (1)$$

calée sur les dimensions du dispositif. La courbure méridienne est nulle ($z'' = 0$).

Modèle membrane élastique: La surface est solution de

$$T\Delta z = 0 \quad (2)$$

donnant

$$z(r) = A \ln r + B \quad (3)$$

avec

$$A = h / \ln(R/r_{\text{sph}}) > 0 \quad (4)$$

. La pente

$$z' = A/r \quad (5)$$

et la courbure

$$z'' = -A/r^2 \neq 0 \quad (6)$$

enrichissent la réaction normale par rapport au modèle conique (voir Annexe).

Intégration numérique

Afin de résoudre les équations différentielles, il est nécessaire de procéder à une intégration numérique du système. Elles sont résolues par quatre méthodes. Le choix d'un intégrateur résulte d'un compromis entre ordre de précision, conservation de l'énergie à long terme et coût par pas.

Les schémas sont définis avec $\mathbf{v}_n = (\dot{r}_n, \dot{\theta}_n)$ et $\mathbf{a}_n = (\ddot{r}_n, \ddot{\theta}_n)$ à t_n :

Euler explicite (ordre 1, non-symplectique):

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \Delta t \mathbf{a}_n, \quad \mathbf{r}_{n+1} = \mathbf{r}_n + \Delta t \mathbf{v}_n$$

Euler semi-implicite (ordre 1, symplectique):

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \Delta t \mathbf{a}_n, \quad \mathbf{r}_{n+1} = \mathbf{r}_n + \Delta t \mathbf{v}_{n+1}$$

Velocity-Verlet (ordre 2, symplectique):

$$\begin{aligned} \mathbf{r}_{n+1} &= \mathbf{r}_n + \Delta t \mathbf{v}_n + \frac{\Delta t^2}{2} \mathbf{a}_n, \\ \mathbf{v}_{n+1} &= \mathbf{v}_n + \frac{\Delta t}{2} (\mathbf{a}_n + \mathbf{a}_{n+1}) \end{aligned}$$

Runge-Kutta 4 (ordre 4, non-symplectique): évaluation de quatre pentes $k_1, \dots, k_4$ à des sous-pas intermédiaires, combinées selon $\mathbf{q}_{n+1} = \mathbf{q}_n + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4)$.

Sur le plan numérique, l'intégration repose sur des systèmes discrets à pas de temps fini, ce qui introduit des erreurs de troncature et d'arrondi. La singularité en $r = 0$ est évitée par une condition d'arrêt physique: la simulation est stoppée dès que $r \leq r_{\min}$ (collision avec la sphère centrale) ou $r \geq R$ (sortie du bord).

Hypothèses

Afin de rendre le problème réalisable analytiquement et numériquement, plusieurs hypothèses simplificatrices ont été introduites. Ces hypothèses permettent de réduire la complexité du système physique réel tout en conservant les mécanismes de la dynamique.

Par ailleurs, plusieurs effets physiques sont négligés dans les deux modèles: les frottements, les pertes d'énergie, ainsi que les éventuelles déformations dynamiques de la surface. La masse du mobile est supposée ponctuelle et la surface parfaitement symétrique autour de l'axe de révolution. Les hypothèses omettent également les erreurs de modélisation qui comprennent des erreurs de mesures liées au manque de précision des instruments de mesure.

Ces simplifications, ces hypothèses constituent un compromis entre fidélité physique et simplicité mathématique. Elles permettent de comparer deux niveaux de modélisation: une approximation géométrique simple (modèle conique) et une description plus riche intégrant la courbure (modèle de membrane élastique).

Modèles d'apprentissage automatique

Une seconde classe de méthodes de simulation repose sur l'utilisation de techniques d'apprentissage automatique, visant à approximer directement la dynamique du système à partir de données. Contrairement aux approches déterministes fondées sur la modélisation explicite des lois physiques, ces méthodes s'inscrivent dans un cadre empirique, dans lequel la relation entrée-sortie est apprise à partir d'un ensemble fini d'observations. Dans cette étude, les données générées par les simulations physiques sont utilisées comme ensemble d'entraînement. L'objectif du ML est d'apprendre une fonction

$$f : \mathbb{R}^4 \rightarrow \mathbb{R}^{2N}$$

telle que:

$$\mathbf{y} = f(\mathbf{x}),$$

où $\mathbf{x}$ représente les conditions initiales du système et $\mathbf{y}$ la trajectoire correspondante.

Plus précisément, le vecteur d'entrée est défini par:

$$\mathbf{x} = (x_0, y_0, v_{x0}, v_{y0}) \in \mathbb{R}^4,$$

où x_0 et y_0 désignent les coordonnées initiales de la bille, et v_{x0} , v_{y0} les composantes initiales de sa vitesse.

La sortie correspond à une discrétisation temporelle de la trajectoire:

$$\mathbf{y} = (x_1, y_1, x_2, y_2, \dots, x_N, y_N) \in \mathbb{R}^{2N},$$

où N désigne le nombre de pas de temps considérés. Cette formulation permet d'exprimer la dynamique du système sous la forme d'une application entre vecteurs d'entrée et de sortie.

Deux modèles ont été considérés pour approximer la fonction f : la régression linéaire (LR) et le perceptron multicouche (MLP).

Regression linéaire

La régression linéaire constitue un modèle paramétrique dans lequel la sortie est exprimée comme une combinaison affine des variables d'entrée [1]. Dans ce cas, elle est utilisée pour approximer la fonction $f : \mathbb{R}^4 \rightarrow \mathbb{R}^{2N}$ reliant les conditions initiales à la trajectoire de la bille.

Plus précisément, le modèle s'écrit:

$$\hat{\mathbf{y}} = \mathbf{W}\mathbf{x} + \mathbf{b}, \quad \mathbf{W} \in \mathbb{R}^{2N \times 4}, \mathbf{b} \in \mathbb{R}^{2N}.$$

Chaque composante de la trajectoire prédite est ainsi modélisée comme une combinaison linéaire des conditions initiales. Toutefois, un tel modèle ne permet pas de capturer les dépendances temporelles entre les différents instants, ni les comportements non linéaires de la trajectoire. Par conséquent, sa capacité de modélisation reste limitée pour des dynamiques complexes.

Multicouche perceptron

Dans le cas d'une architecture à une couche cachée, la fonction f s'écrit:

$$f(\mathbf{x}) = \mathbf{W}^{(2)} \sigma(\mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)},$$

où σ est une fonction d'activation non linéaire. σ est représentée par la fonction ReLU (Rectified Linear Unit), définie par $\sigma(z) = \max(0, z)$ [1] [2].

Les paramètres du modèle, sont estimés en minimisant une fonction de coût, généralement choisie comme l'erreur quadratique moyenne (MSE) (voir l'Annexe). M désigne le nombre d'échantillons d'entraînement, pour chaque exemple i , $\mathbf{y}^{(i)}$ correspond à la trajectoire réelle associée aux conditions initiales $\mathbf{x}^{(i)}$, et $\hat{\mathbf{y}}^{(i)} = f_{\theta}(\mathbf{x}^{(i)})$ la prédiction du modèle.

Validité des modèles de ML

Afin d'évaluer la capacité de généralisation des modèles d'apprentissage automatique, une procédure de validation croisée à K -folds ($K = 5$) a été adoptée [3] [1]. La performance finale est estimée par la moyenne des erreurs obtenues sur chacun des plis.

Le MLP est entraîné sur 50 epochs à chaque pli. La métrique d'évaluation choisie est la RMSE sur le rayon (voir l'Annexe). r_i désigne la valeur réelle du rayon à l'instant i et $\hat{r}_i$ la valeur prédite par le modèle, évalués sur 64 trajectoires de tests générées.

Les résultats obtenus sont résumés dans l'Annexe 3.

Les résultats montrent que le MLP surpasse nettement le modèle linéaire, avec une RMSE moyenne inférieure de 35 %, indiquant une meilleure modélisation des dynamiques non linéaires. Le modèle linéaire présente un écart-type quasi nul ($\sigma \approx 10^{-5}$ m), signe d'une grande stabilité mais d'un biais élevé limitant sa capacité d'adaptation. À l'inverse, le MLP affiche une variabilité plus importante ($\sigma = 1.36 \times 10^{-3}$ m), reflétant une sensibilité aux données. Malgré cela, il reste performant même dans le pire cas et surpasse systématiquement le modèle linéaire, ce qui confirme la robustesse de ses performances. Ces résultats sont illustrés dans la figure 2 et la figure 3

Importance de la taille du jeu de données

La figure 2 met en évidence l'influence de la taille du jeu de données sur la précision des modèles. Pour un nombre très limité d'expériences, des écarts importants entre les trajectoires prédites et la trajectoire réelle sont observés, traduisant un sous-apprentissage. L'augmentation du nombre d'expériences conduit à une amélioration significative de la qualité des prédictions. Le MLP converge progressivement vers la trajectoire réelle et capture plus fidèlement les dynamiques non linéaires, tandis que le modèle linéaire présente une amélioration plus limitée, contrainte par sa capacité de représentation. Ces résultats indiquent que l'accroissement du volume de données permet de réduire l'erreur de prédiction et d'améliorer la stabilité des modèles, avec un effet particulièrement prononcé pour le MLP.

Calcul numérique

Les sous-sections suivantes sont dédiées à expliquer les différents types d'erreurs pris en compte dans nos analyses. Les formalismes mathématiques sont détaillés en annexe.

Erreurs numériques

Dans le cadre d'une approche scientifique, il est important de prendre en compte tous les paramètres pouvant entrer en jeu et potentiellement perturber et modifier l'expérience. Les erreurs de modélisation ont déjà été abordées, toutefois, elles ne sont pas les seules à intervenir. En particulier, il faut considérer les erreurs numériques qui apparaissent lorsque l'ordinateur effectue des calculs [4, p. 16].

Erreurs de troncature Les modèles itératifs à pas discret utilisés sont infinis. Il est donc impossible d'obtenir une solution exacte en un temps fini même en faisant abstraction de tout autre type d'erreur pouvant entrer en jeu. Afin d'obtenir un résultat, il faut établir une variable n_steps qui limite le nombre d'itération de manière à obtenir un bon équilibre entre temps d'exécution et précision du résultat.

Erreurs d'arrondi Les erreurs d'arrondi sont des erreurs causées par la représentation finie des nombres en machine dans les registres du processeur (généralement 32 ou 64 bits). L'ordinateur ne peut pas représenter tous les nombres réels avec une précision infinie, ce qui crée alors une approximation. Nous concluons que ce type d'erreur apparaît également dans les simulations.

Erreurs de propagation et génération Les erreurs de propagation et de génération sont des erreurs qui apparaissent lorsque les erreurs numériques se propagent à travers les calculs. Les erreurs de génération se produisent à chaque itération, étape des calculs et les erreurs de propagation regroupent toutes les erreurs de génération jusqu'à l'étape courante.

Analyse numérique du problème

Les notions de consistance, stabilité et de conditionnement sont utilisées pour analyser les propriétés de nos modèles. Ce sont des propriétés souhaitées pour un algorithme numérique. Par ces analyses, nous pouvons prédire certains comportements de nos modèles lors des simulations. Les formalismes mathématiques sont en annexe.

Consistance La consistance est la capacité d'un algorithme à produire des résultats qui convergent vers la solution exacte du problème. Dans notre cas, la méthode de Euler (explicite ou implicite) est consistante.

Stabilité La stabilité est la propriété d'un algorithme qui garantit que les erreurs apportées durant les calculs ne s'amplifient pas de manière exponentielle au fil des itérations.

Convergence La notion de convergence d'un algorithme numérique permet d'étudier les itérations lorsque celles-ci tendent vers l'infini. En particulier, une méthode dite convergente va converger vers la solution tandis qu'une méthode divergente s'en éloigne.

Modèle	Précision des résultats numériques (40%)	Hypothèses simplifiantes (30%)	Temps de calcul (10%)	Accessibilité du modèle (20%)	Score
Modèle conique	3/10	7/10	10/10	6/10	5,5/10
Modèle membrane élastique	3/10	8/10	10/10	5/10	5,6/10
Régression linéaire	8/10	2/10	4/10	7/10	5,6/10
MLP	9/10	2/10	2/10	7/10	5,8/10

Table 1: Évaluation multicritère des modèles

Conditionnement Le conditionnement décrit la sensibilité intrinsèque du problème: une petite perturbation des données d’entrée peut produire une petite ou une grande variation de la solution exacte lors de la résolution numérique. Un problème bien conditionné est peu sensible aux perturbations, tandis qu’un problème mal conditionné les amplifie.

Comparaison des méthodes d’approximation numérique

Plusieurs méthodes d’approximation numérique peuvent être utilisées pour résoudre les équations différentielles ordinaires associées aux différents modèles étudiés. Le tableau suivant présente, pour chaque méthode et chaque modèle, l’erreur géométrique normalisée mesurant l’écart entre la trajectoire simulée et la trajectoire réelle.

Nous tenons à faire certaines précisions au sujet de ces valeurs. En effet, ces dernières sont très similaires et les faibles différences sont de l’ordre du million-ième. En plus de leur similitude, ces résultats sont très élevés indiquant ainsi des erreurs de précision numériques importantes.

Nous en concluons deux choses. Premièrement, les modèles physiques sont peu précis dans un cadre de simulation. D’autre part, le tableau 2 montre qu’il n’y a aucune différence à notre échelle pour le choix de l’intégrateur.

Conclusion et discussion

Dans le cadre de ce travail, la comparaison des différents modèles de simulation ne peut se limiter à un unique critère de performance. En effet, les approches considérées présentent des caractéristiques hétérogènes, tant du point de vue de la fidélité physique que des propriétés numériques ou de leur mise en œuvre. Il apparaît ainsi nécessaire d’introduire un cadre d’analyse permettant de confronter ces modèles selon plusieurs dimensions pertinentes.

À cette fin, une démarche d’évaluation multicritère est adoptée. Celle-ci repose sur l’identification de quatre axes principaux:

- la précision des résultats, évaluée à travers les erreurs numériques (40%)
- les hypothèses simplifiantes considérées et leur quantité (30%)
- le temps requis pour effectuer les simulations (10%)
- l’accessibilité du modèle, incluant à la fois sa simplicité de mise en œuvre au niveau technique et sa capacité à être interprété ou vulgarisé (20%)

Une telle approche permet de construire un indicateur synthétique facilitant la comparaison globale des modèles, tout en conservant une lecture structurée des compromis qu’ils impliquent. Elle s’inscrit dans une logique d’aide à la décision plutôt que dans une volonté d’établir un classement absolu.

En attribuant les notes correspondantes à chaque modèle selon ces critères, les résultats obtenus se trouvent dans le tableau ci-dessus.

Nous en concluons que le meilleur modèle est le MLP.

Nous soulignons toutefois que cette pondération repose sur des choix méthodologiques propres à ce travail subjectifs. Elle a pour but de regrouper tous les aspects rencontrés au cours de ce travail, notamment l’aspect de vulgarisation qui représentait un aspect majeur lors de la présentation au Printemps des Sciences. La pondération permet également d’équilibrer la partie de subjectivité que nous avons essayé de minimiser lors de l’attribution des notes aux travers de discussion interne à notre groupe. Dans ce sens, l’évaluation proposée doit être comprise comme qualitative et indicative: elle vise à structurer l’analyse et à expliciter les compromis entre modèles, sans prétendre à une objectivité stricte ni à une généralisation indépendante du contexte d’étude.

Annexes

Nous détaillons ici les éléments qui ne sont pas indispensables à la compréhension globale de notre travail. Ces formalismes sont dédiés aux lecteurs souhaitant approfondir leur compréhension.

Équations physiques

Notions préliminaires Les termes techniques suivants sont utilisés dans la modélisation.

Contrainte: En mécanique analytique, une contrainte est une relation géométrique ou cinématique qui restreint les configurations accessibles d'un système. Elle réduit le nombre de degrés de liberté effectifs et doit être prise en compte dans la formulation des équations du mouvement.

Contrainte holonome: Une contrainte est dite holonome si elle s'exprime uniquement en termes de coordonnées (et éventuellement du temps), sans faire intervenir les vitesses de façon non intégrable. Formellement: $f(q_1, \dots, q_n, t) = 0$. Dans notre cas, $z = z(r)$ est holonome: la bille est contrainte à rester sur la surface, ce qui réduit les trois coordonnées cartésiennes (x, y, z) à deux degrés de liberté (r, θ) .

Surface de révolution: Surface engendrée par la rotation d'une courbe méridienne $z = z(r)$ autour de l'axe vertical. La symétrie axiale implique que la dynamique ne dépend que de r et θ , et que le moment cinétique $L = mr^2\dot{\theta}$ est conservé en l'absence de frottement.

Coordonnées généralisées: Variables indépendantes paramétrisant l'espace des configurations d'un système soumis à des contraintes. Pour notre surface de révolution, (r, θ) sont les coordonnées généralisées ; l'état complet du système est $\mathbf{q} = (r, \theta, \dot{r}, \dot{\theta}) \in \mathbb{R}^4$.

Courbure méridienne: Courbure de la courbe $z(r)$ dans le plan méridien (plan contenant l'axe de révolution). Elle quantifie la variation de pente le long de la surface et vaut:

$$\kappa = \frac{z''}{(1 + z'^2)^{3/2}} \quad (7)$$

où $z' = dz/dr$ et $z'' = d^2z/dr^2$. Pour le cône ($z'' = 0$), $\kappa = 0$: la surface est développable et n'exerce aucune force normale supplémentaire due à la trajectoire. Pour la membrane élastique ($z'' < 0$), $\kappa \neq 0$: la courbure modifie la réaction normale et donc le frottement.

Force généralisée. Analogie d'une force dans l'espace des coordonnées généralisées. Pour une force réelle $\vec{F}$ s'appliquant en un point de coordonnées $(q_1, \dots, q_n)$, la force généralisée associée à q_i est

$Q_i = \vec{F} \cdot \partial \vec{r} / \partial q_i$. Dans notre cas, Q_r et Q_θ sont les projections du frottement de Coulomb sur les directions radiale et angulaire de la surface.

Intégrateur symplectique. Schéma numérique qui préserve la structure symplectique de l'espace des phases (volume de l'espace (q, p)). Cette propriété garantit la conservation approchée de l'énergie sur de longues durées, sans dérive systématique, contrairement aux schémas non-symplectiques comme Euler explicite ou RK4.

Cadre lagrangien commun Les deux modèles partagent la même structure. La surface étant de révolution, la contrainte holonome $z = z(r)$ réduit la dynamique à (r, θ) . L'énergie cinétique sur la surface vaut:

$$T = \frac{1}{2}m \left[\dot{r}^2(1 + z'^2) + r^2\dot{\theta}^2 \right]. \quad (8)$$

Les équations d'Euler-Lagrange avec forces de dissipation Q_r, Q_θ donnent le système:

$$\ddot{r} = \frac{r\dot{\theta}^2 - gz' - \dot{r}^2 z' z'' + Q_r/m}{1 + z'^2}, \quad (9)$$

$$\ddot{\theta} = \frac{-2\dot{r}\dot{\theta}}{r} + \frac{Q_\theta}{mr^2}. \quad (10)$$

Sans frottement, $L = mr^2\dot{\theta}$ est une constante du mouvement. La conservation de l'énergie mécanique

$$E = \frac{1}{2}m \left[\dot{r}^2(1 + z'^2) + r^2\dot{\theta}^2 \right] + mgz(r) \quad (11)$$

sert de critère de validation numérique (dérive $|\Delta E/E_0|$).

Cadre physique du cône Pente constante calée sur les dimensions du dispositif:

$$\frac{dz}{dr} = \tan \alpha = \frac{h}{R - r_{\text{sph}}} \quad (12)$$

$$z(r) = \tan \alpha \quad (13)$$

$$r - r_{\text{sph}}, \quad z'' = 0 \quad (14)$$

L'accélération radiale effective est $-gz' = -g \tan \alpha$ (constante). La condition d'orbite stable ($\ddot{r} = 0, \dot{r} = 0$) donne $\dot{\theta}_{\text{orb}} = \sqrt{g \tan \alpha / r}$.

Cadre physique de la membrane élastique Solution de $T\Delta z = 0$ en coordonnées polaires, calée sur $z(r_{\text{sph}}) = 0$ et $z(R) = h$:

$$z(r) = A \ln r + B, \quad (15)$$

$$z'(r) = \frac{A}{r}, \quad (16)$$

$$z''(r) = -\frac{A}{r^2}, \quad (17)$$

avec $A = h / \ln(R/r_{\text{sph}}) > 0$, $B = -A \ln r_{\text{sph}}$. La tension de membrane est déterminée par l'équilibre vertical en r_{sph} : $T \cdot 2\pi r_{\text{sph}} |z'(r_{\text{sph}})| = Mg$, soit $T = Mg / (2\pi A)$. La dépendance en $1/r$ (contre $1/r^2$ pour la membrane élastique) assure une pente finie et modérée sur tout le domaine ; le terme $\dot{r}^2 z' z''$ dans (9) capture la courbure méridienne $\kappa = z'' / (1 + z'^2)^{3/2} \neq 0$.

Forces généralisées de frottement de Coulomb La vitesse sur la surface et la réaction normale sont:

$$v = \sqrt{\dot{r}^2 (1 + z'^2) + r^2 \dot{\theta}^2} \quad (18)$$

$$N = m \sqrt{(g \cos \alpha_{\text{loc}})^2 + (v^2 \kappa)^2} \quad (19)$$

où $\kappa = z'' / (1 + z'^2)^{3/2}$ est la courbure méridienne. Les forces généralisées de Coulomb valent:

$$Q_r = -\mu N \frac{\dot{r} \sqrt{1 + z'^2}}{v}, \quad (20)$$

$$Q_\theta = -\mu N \frac{r \dot{\theta}}{v} \quad (21)$$

Métriques d'erreur en machine learning

$$MSE = \frac{1}{M} \sum_{i=1}^M \left\| \mathbf{y}^{(i)} - \hat{\mathbf{y}}^{(i)} \right\|_2^2, \quad (22)$$

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (r_i - \hat{r}_i)^2} \quad (23)$$

Erreurs numériques: formellement

Ici sont définis formellement les types d'erreurs à l'aide d'équations mathématiques afin d'approfondir les explications des sections dédiées aux erreurs numériques (voir Calcul numérique).

Erreurs d'arrondi Pour définir formellement ce type d'erreur, notons par $\mathbb{F}(b, t, L, U)$ l'ensemble des nombres réels qui sont représentés par une notation à virgule flottante en base b , comportant t chiffres significatifs et dont l'exposant varie dans l'intervalle $[L, U]$. Considérons un ordinateur utilisant cette notation $\mathbb{F}$. Soit $x \in \mathbb{R}$ un nombre réel quelconque et $fl(x) \in \mathbb{F}$ la représentation machine à virgule flottante de x . Désignons par ε_x son erreur relative

$$\varepsilon_x = \frac{|fl(x) - x|}{|x|} \quad (24)$$

Evaluons la borne supérieure de l'erreur relative d'arrondi. Il est évident que cette quantité ne peut

dépasser la moitié de la distance relative entre les deux nombres consécutifs à virgule flottante $x_i \in \mathbb{F}(b, t, L, U)$ et $x_{i+1} \in \mathbb{F}(b, t, L, U)$ tels que

$$x_i \leq x \leq x_{i+1} \quad (25)$$

Il vient alors que

$$\varepsilon_x \equiv |\rho_x| \leq \frac{1}{2} \left| \frac{x_{i+1} - x_i}{x} \right| \leq \frac{1}{2} b^{1-t} = \frac{1}{2} \epsilon = u, \quad (26)$$

où u est dite la précision machine.

Avec la notion d'écart relatif définie telle que

$$\rho_x = \frac{\hat{x} - x}{x} \quad (27)$$

il en résulte

$$\hat{x} = fl(x) = x(1 + \rho_x), \quad (28)$$

où $\varepsilon_x = |\rho_x| \leq u$ est dite l'erreur d'arrondi [4].

Analyse numérique du problème: formellement

Les notions de consistance, consistence, convergence et de conditionnement décrites dans la section Analyse numérique du problème sont détaillées dans les sous-sections suivantes.

La notion d'algorithme Nous définissons la notion d'algorithme numérique. Étant donné un problème bien posé $F(x, d) = 0$, nous définissons l'algorithme numérique pour la résolution du problème F par la suite de problèmes approchés

$$F_1(x^{(1)}, d^{(1)}), F_2(x^{(2)}, d^{(2)}), \dots, F_n(x^{(n)}, d^{(n)})$$

dépendant d'un paramètre n , où $x^{(n)}$ est la solution du sous-problème F_n [4].

Consistance Un algorithme $F_n(x^{(n)}, d^{(n)})$ est dit consistant si

$$\lim_{n \rightarrow \infty} F_n(x, d^{(n)}) = F(x, d) = 0, \quad (29)$$

c.-à-d., si la solution exacte x du problème est une solution de $F_n(x^{(n)}, d^{(n)}) = 0$ pour $n \rightarrow \infty$.

En d'autres termes, un algorithme est consistant si à partir d'une certaine étape le sous-problème approché F_n a la solution x parmi ses solutions [4].

Stabilité En particulier, un algorithme numérique est dit stable (ou bien posé) si

1. il existe pour tout n une solution unique $x^{(n)}$;
2. pour tout $\epsilon > 0$, il existe $\eta > 0$ et n_0 tels que

$$\begin{aligned} \|\delta d^{(i)}\| &\leq \eta \quad \forall i = 1, \dots, m \\ \Rightarrow \|\bar{x}^{(n)} - x^{(n)}\| &\leq \epsilon \end{aligned}$$

En d'autres termes, un algorithme est stable si des petites perturbations des données du sous-problème F_n entraînent de petites perturbations de sa solution $x^{(n)}$.

Convergence Le théorème d'équivalence de Lax [4] stipule logiquement que pour un problème bien posé, la consistance couplée à la stabilité assure la convergence globale du modèle discret vers la vérité continue.

Toutefois, ce théorème ne s'extrapole pas aux réseaux de neurones (MLP, LR). Entraînés de manière isolée pour lisser l'étape $n \rightarrow n + 1$, la régression est aveugle aux variations sur temps long. Intégrée en boucle auto-régressive (*step-by-step*), ses légères imprécisions déplacent l'état évalué hors du domaine vu à l'apprentissage (un *distribution shift*). Ces erreurs alimentent exponentiellement les futures imprécisions, déclenchant sur la durée une déviation divergente similaire à de l'instabilité numérique classique.

Conditionnement Le conditionnement d'un problème de la forme $x = F(d)$ mesure la sensibilité de la solution x du problème aux changements des données d . Nous distinguons deux types de conditionnement:

- **le conditionnement relatif:** utilisé de manière générale
- **le conditionnement absolu:** utilisé lorsque les données $d = 0$ ou quand la solution $x = 0$

Nous ne détaillons ici que le conditionnement relatif.

En particulier ([4]), soient δd une perturbation admissible des données et δx la modification induite sur la solution du problème $x = F(d)$. On appelle conditionnement relatif de ce problème la quantité

$$\kappa(d) = \lim_{|D| \rightarrow 0} \sup_{\delta d \in D} \frac{\|\delta x\| / \|x\|}{\|\delta d\| / \|d\|} \quad (30)$$

où $\sup$ dénote la borne supérieure et D est un voisinage de l'origine.

Si $F(\cdot)$ est différentiable en d et si nous notons sa dérivée par

$$F'(d) = \lim_{\delta d \rightarrow 0} \frac{\delta x}{\delta d},$$

on obtient

$$\kappa(d) = \lim_{\delta d \rightarrow 0} \frac{\|\delta x\|}{\|\delta d\|} \frac{\|d\|}{\|x\|} = \|F'(d)\| \frac{\|d\|}{\|F(d)\|} \quad (31)$$

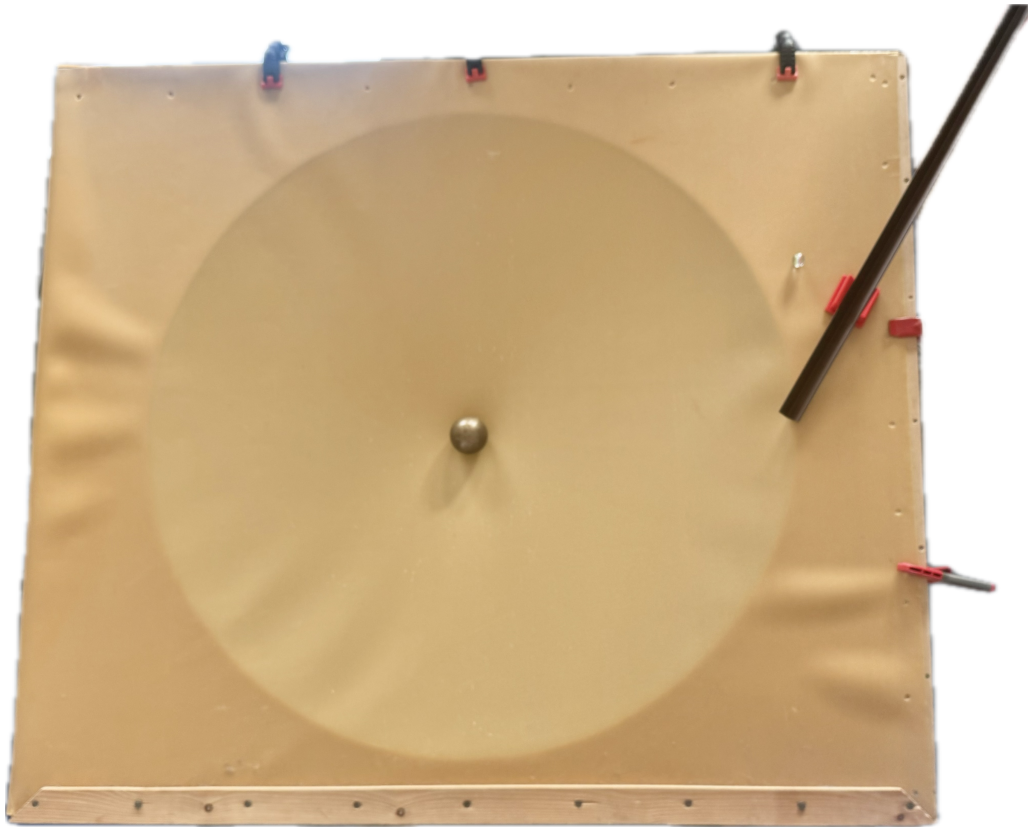
où le symbole $\|\cdot\|$ désigne la norme matricielle.

En outre, il est important de remarquer que:

- un problème mal posé est forcément aussi un problème mal conditionné, mais un problème mal conditionné n'est pas forcément mal posé
- le fait d'être bien conditionné est une propriété du problème qui est indépendante de la méthode choisie pour le résoudre

Table 2: Erreur géométrique normalisée entre la trajectoire simulée et la trajectoire réelle pour chaque combinaison de modèle et de méthode numérique.

Méthode	Conique	Membrane
Euler exp.	0.7681	0.7682
Euler semi-imp.	0.7681	0.7682
RK4	0.7682	0.7682

**Figure 1:** Photo du dispositif expérimental utilisé pour les lancements de la bille.

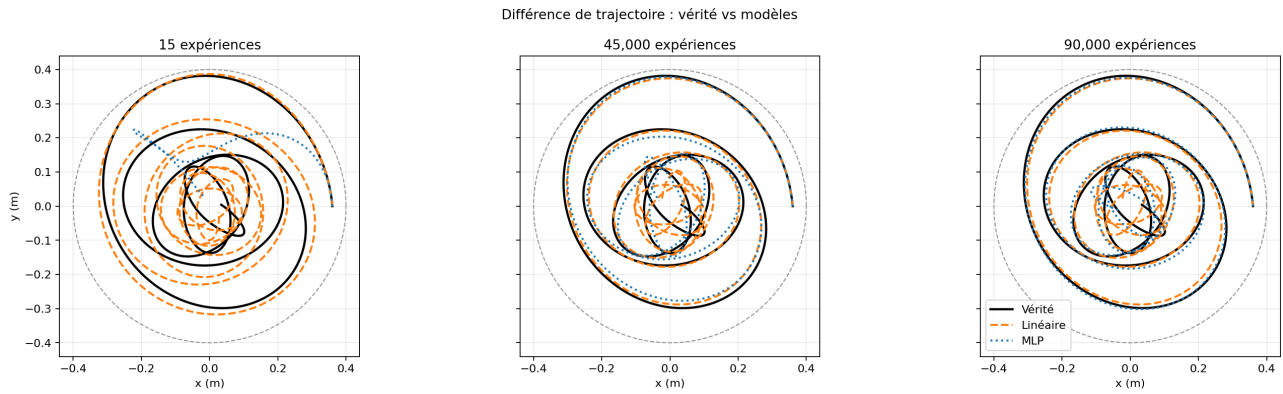


Figure 2: Différence entre la vraie trajectoire et les modèles linéaire et MLP pour trois tailles de jeu de données (15, 45000, 90000).

Validation croisée 5-fold — 90 000 expériences

Linéaire — RMSE rayon moyen

0.00841 m

$\sigma = 0.00001$

MLP — RMSE rayon moyen

0.00565 m

$\sigma = 0.00123$

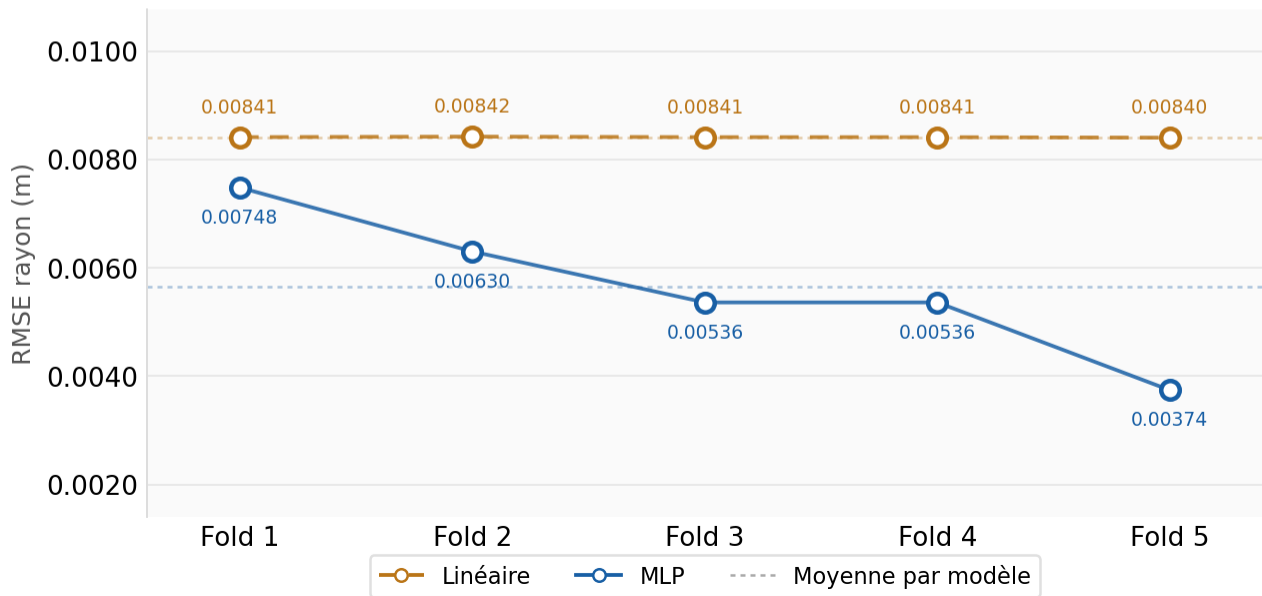


Figure 3: Cross-validation avec K-fold (K=5) pour les modèles regression linéaire et MLP.

References

- [1] Université Libre de Bruxelles. *Statistical Foundations of Machine Learning*. INFO-F422. 2025. URL: https://ulb-mlg.github.io/INFO-F422/practicals/_build/html/03.html.
- [2] scikit-learn developers. *Neural network models (supervised)*. scikit-learn 1.8.0 documentation. 2026. URL: https://scikit-learn.org/stable/modules/neural_networks_supervised.html.
- [3] GeeksforGeeks. *Cross Validation in Machine Learning*. Sept. 2025. URL: <https://www.geeksforgeeks.org/machine-learning/cross-validation-machine-learning/>.
- [4] Maarten Jansen et al. *SYLLABUS Cours de Calcul Formel et Numérique*. INFO-F-205. Belgique, Feb. 2022.
- [5] Gianluca Bontempi. *SYLLABUS Cours de Modélisation et Simulation*. INFO-F-305. Belgique, July 2024.
- [6] John C. Strikwerda. *Finite Difference Schemes and Partial Differential Equations*. 2nd. Philadelphia, PA: SIAM, 2004. ISBN: 0898715679.
- [7] Wikipedia contributors. *Lagrangian mechanics*. Mar. 2026. URL: https://en.wikipedia.org/wiki/Lagrangian_mechanics.
- [8] Wikipedia contributors. *Euler-Lagrange equations*. Apr. 2026. URL: https://en.wikipedia.org/wiki/Euler%E2%80%93Lagrange_equation.